Student: Nazila Hoche Abdi
SID: 40290161

SPRINT 2 AI LOGS

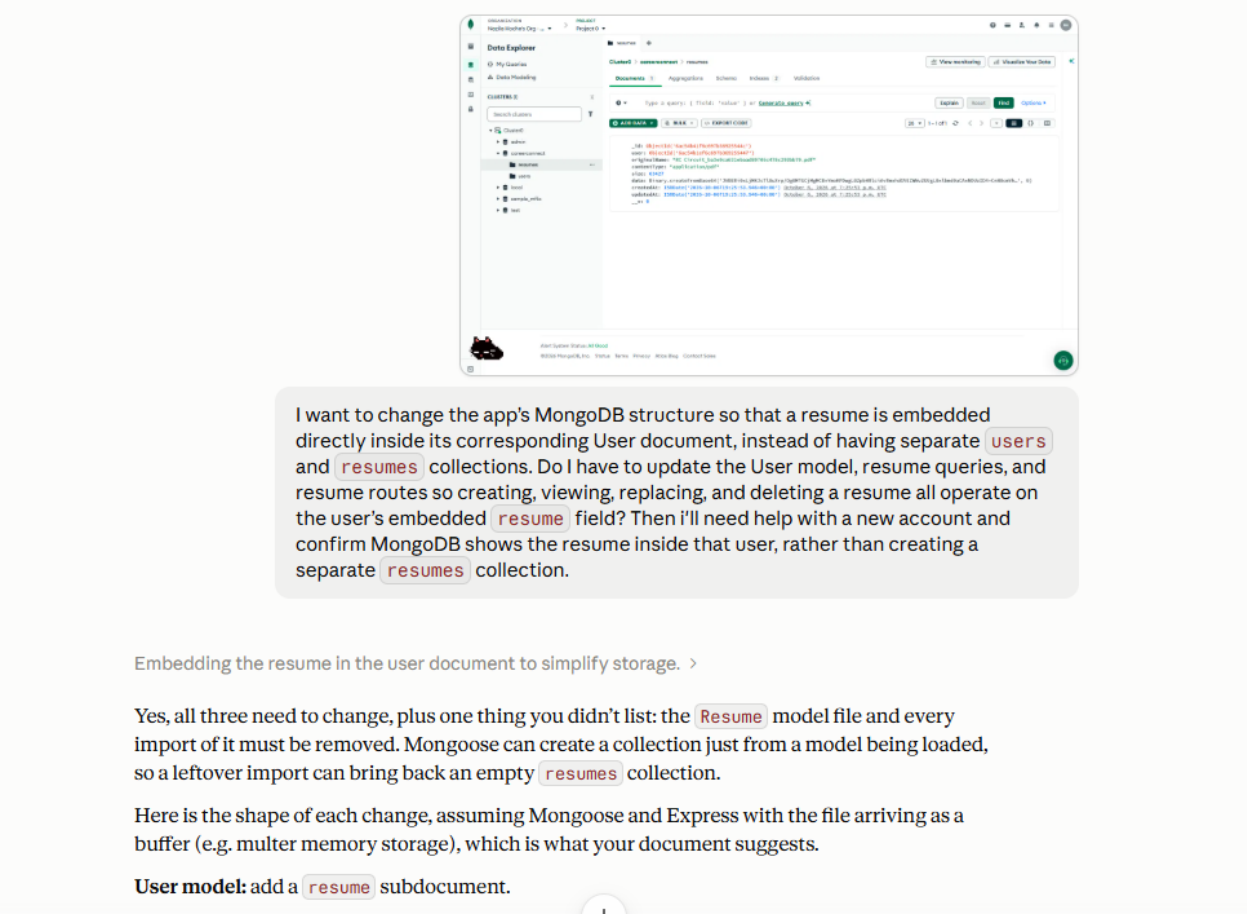
Task ID: SCRUM-158: Fix database issue

Tool used: Claude

Purpose of AI Use: To understand how to store each resume inside its user document instead of in a separate resumes collection.

Prompt: I want to change the app's MongoDB structure so that a resume is embedded directly inside its corresponding User document, instead of having separate users and resumes collections. What do I need to update in the User model, resume queries, and resume routes?

Response: AI explained that the User model needed an embedded resume field, the queries and routes needed to read and write that field instead of a Resume model, and the old Resume model had to be removed so the resumes collection would not be recreated.



AI-Suggested Content: An embedded resume subdocument in the User model with the file data excluded from normal queries, rewritten resume queries and routes, and steps to delete the old model and drop the old collection.

Validation: Replaced the files, restarted the server, registered a new account, uploaded a resume through the frontend, and confirmed in MongoDB Atlas that the resume was stored inside the user document.

Decision: Accepted after testing.

Reflection: AI helped me understand embedded documents in MongoDB and why leftover model files can recreate old collections.

Responsible Person: Nazila Hoche Abdi

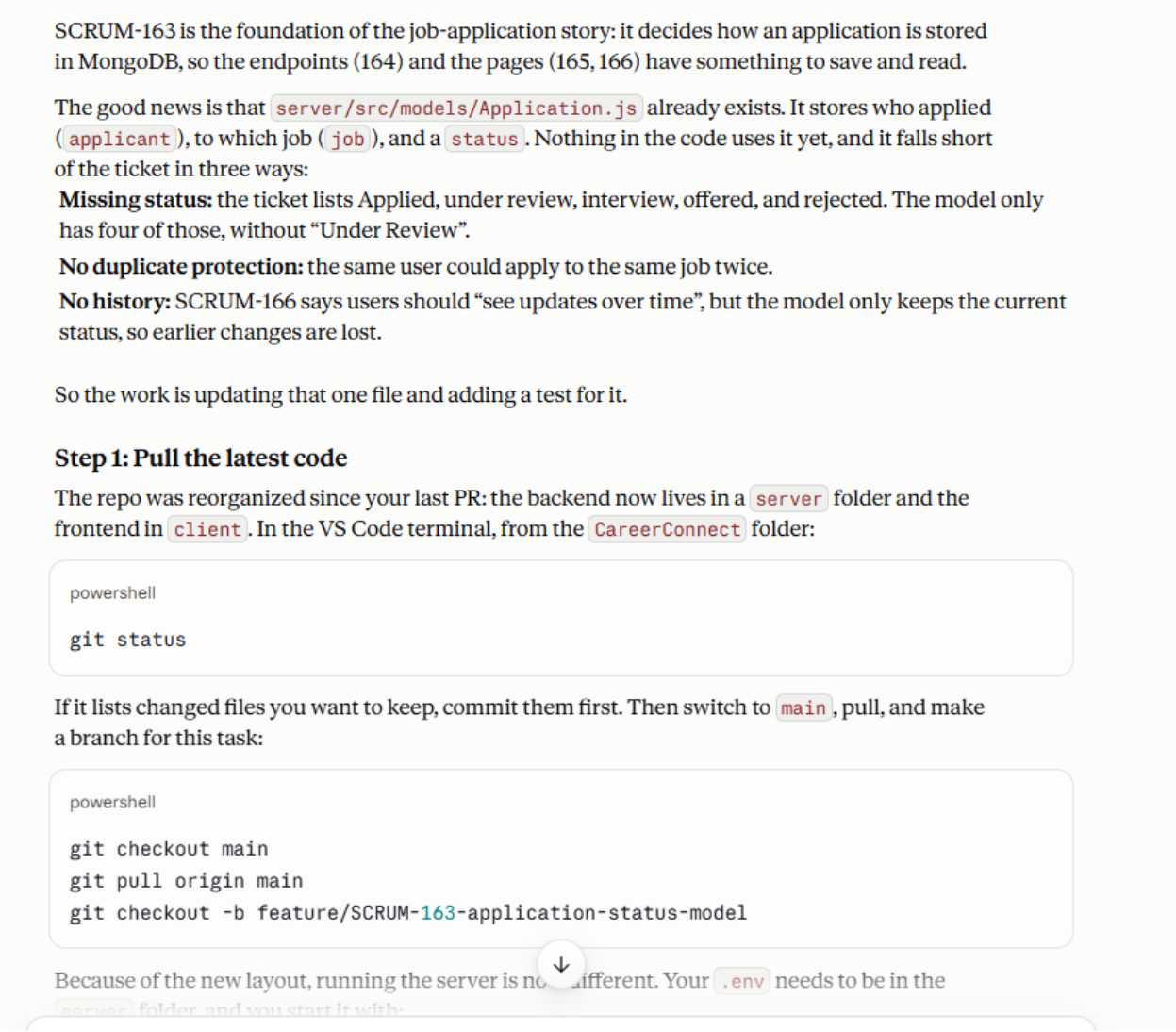
Task ID: SCRUM-163: Add application persistence and status model

Tool used: Claude

Purpose of AI Use: To understand what changes the Application model needed to meet the ticket requirements.

Prompt: Explain SCRUM-163 and walk me through the changes I need to make to the Application model and why.

Response: AI pointed out that the existing model was missing the “Under Review” status, had no history of status changes, and did not prevent duplicate applications.



SCRUM-163 is the foundation of the job-application story: it decides how an application is stored in MongoDB, so the endpoints (164) and the pages (165, 166) have something to save and read.

The good news is that `server/src/models/Application.js` already exists. It stores who applied (`applicant`), to which job (`job`), and a `status`. Nothing in the code uses it yet, and it falls short of the ticket in three ways:

Missing status: the ticket lists Applied, under review, interview, offered, and rejected. The model only has four of those, without “Under Review”.

No duplicate protection: the same user could apply to the same job twice.

No history: SCRUM-166 says users should “see updates over time”, but the model only keeps the current status, so earlier changes are lost.

So the work is updating that one file and adding a test for it.

Step 1: Pull the latest code

The repo was reorganized since your last PR: the backend now lives in a `server` folder and the frontend in `client`. In the VS Code terminal, from the `CareerConnect` folder:

```
powershell
git status
```

If it lists changed files you want to keep, commit them first. Then switch to `main`, pull, and make a branch for this task:

```
powershell
git checkout main
git pull origin main
git checkout -b feature/SCRUM-163-application-status-model
```

Because of the new layout, running the server is no different. Your `.env` needs to be in the server folder and you start it with:

AI-Suggested Content: A shared list of the five statuses, a statusHistory field, a unique index on applicant and job, and unit tests for the model.

Validation: Ran lint and the unit tests (all 13 passed) and reviewed the schema against the ticket requirements.

Decision: Accepted.

Reflection: AI helped me plan the data model so the later endpoints and pages could use it.

Responsible Person: Nazila Hoche Abdi

Task ID: SCRUM-164: Implement application status tracking endpoints

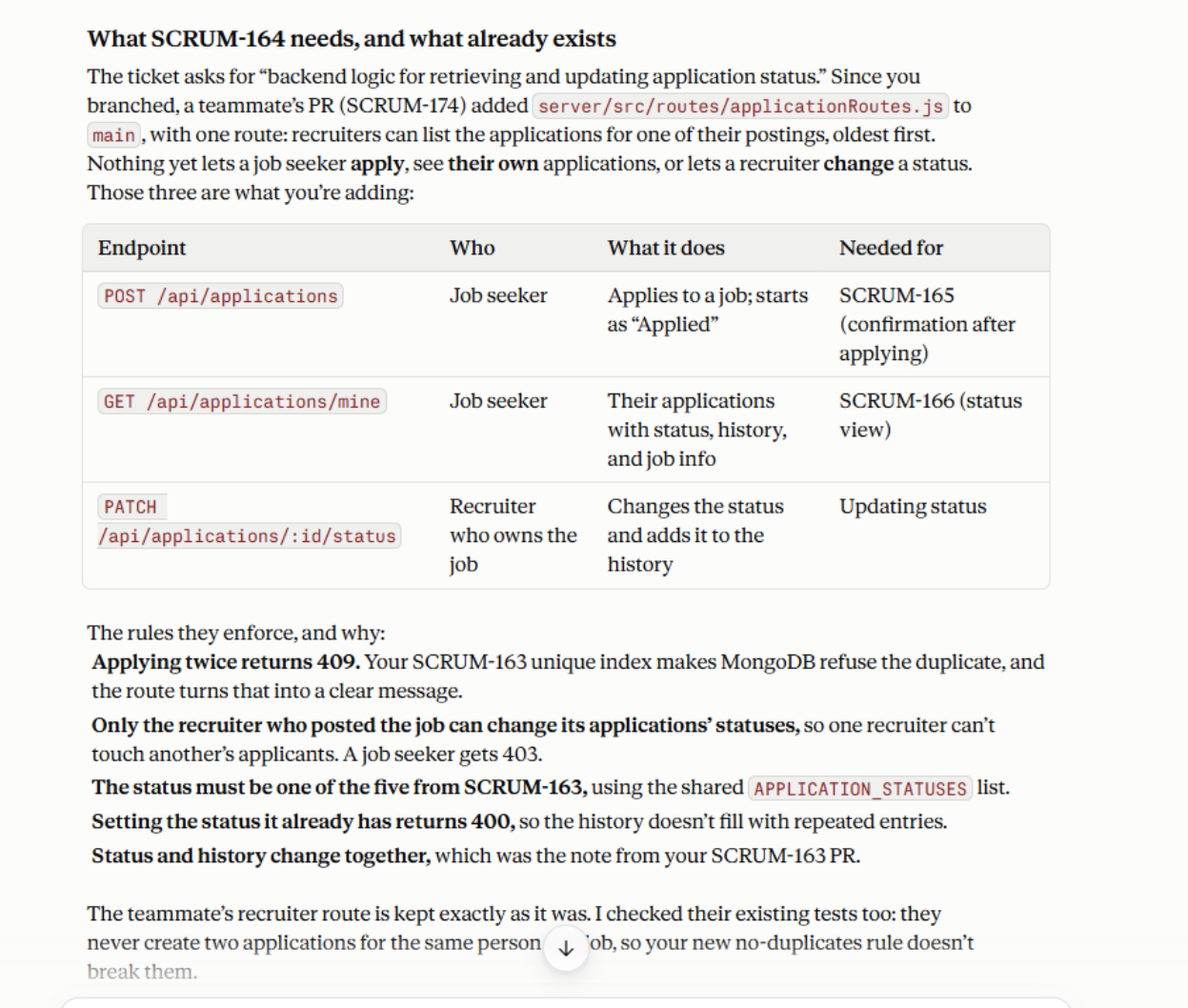
Tool used: Claude

Purpose of AI Use: To design the backend endpoints for applying to jobs and tracking application status.

Prompt: Help me implement SCRUM-164, the endpoints for retrieving and updating application status.

Response: AI proposed three endpoints and the permission rules for each.

AI-Suggested Content: POST /api/applications (job seekers apply), GET /api/applications/mine (job seekers see their applications), PATCH /api/applications/:id/status (the recruiter who owns the job updates the status), plus 17 integration tests.



What SCRUM-164 needs, and what already exists

The ticket asks for "backend logic for retrieving and updating application status." Since you branched, a teammate's PR (SCRUM-174) added `server/src/routes/applicationRoutes.js` to `main`, with one route: recruiters can list the applications for one of their postings, oldest first. Nothing yet lets a job seeker **apply**, see **their own** applications, or lets a recruiter **change** a status. Those three are what you're adding:

Endpoint	Who	What it does	Needed for
`POST /api/applications`	Job seeker	Applies to a job; starts as "Applied"	SCRUM-165 (confirmation after applying)
`GET /api/applications/mine`	Job seeker	Their applications with status, history, and job info	SCRUM-166 (status view)
`PATCH /api/applications/:id/status`	Recruiter who owns the job	Changes the status and adds it to the history	Updating status

The rules they enforce, and why:

Applying twice returns 409. Your SCRUM-163 unique index makes MongoDB refuse the duplicate, and the route turns that into a clear message.

Only the recruiter who posted the job can change its applications' statuses, so one recruiter can't touch another's applicants. A job seeker gets 403.

The status must be one of the five from SCRUM-163, using the shared `APPLICATION_STATUSES` list.

Setting the status it already has returns 400, so the history doesn't fill with repeated entries.

Status and history change together, which was the note from your SCRUM-163 PR.

The teammate's recruiter route is kept exactly as it was. I checked their existing tests too: they never create two applications for the same person [illegible]ob, so your new no-duplicates rule doesn't break them.

Validation: Ran the integration tests. They first failed because of a test timeout and then a lack of disk space on my machine, not because of the code. After fixing both, all tests passed.

Decision: Accepted, with the test setup timeout increased.

Reflection: AI helped me understand role-based permissions and why the user must come from the login token instead of the request body.

Responsible Person: Nazila Hoche Abdi

Task ID: SCRUM-165: Show submission confirmation after apply

Tool used: Claude

Purpose of AI Use: To connect the Apply button to the backend and show a confirmation to the user.

Prompt: Help me implement SCRUM-165, showing a confirmation after a job application is submitted.

Response: AI suggested connecting the Apply button to the new endpoint and showing different content depending on the user.

What SCRUM-165 needs

The ticket: "Display a confirmation message or success state to the user once the job application has been submitted successfully." Right now the job details page has an **Apply Now** button that does nothing. This task makes it call the `POST /api/applications` endpoint from SCRUM-164 and shows a confirmation, so it's connected to the real database, as the parent ticket requires.

What the page now does

Who is looking	What they see
Job seeker, hasn't applied	**Apply Now** button. While sending it says "Submitting..." and can't be clicked twice.
Job seeker, just applied	Green box: **"Application submitted!"**, the job title, company, date, and "Current status: Applied"
Job seeker who already applied (e.g. came back later)	Same box, titled **"You have already applied"**, with the current status from the database
Logged-out visitor	"Log in as a job seeker to apply for this job."
Recruiter	No Apply button, since recruiters can't apply

If the submission fails, the error appears above the button and they can try again. If the server says they already applied (the 409 from SCRUM-164), the page loads that application and shows the "already applied" box instead of an error.

The files

`client\src\api\applicationApi.js` **(new).** Two functions: `applyToJob` calls `POST /api/applications`, and `fetchMyApplications` calls `GET /api/applications/mine`. The second one checks whether you already applied, and SCRUM-166 will reuse it for the status page.

`client\src\components\JobDetail.jsx` **(replace).** The page logic in the table above. Loading the job works exactly as before.

`client\src\components\JobDetail.test.jsx` (n…). Six tests, one per row of the table, plus the error case and the 409 case.

AI-Suggested Content: An applicationApi.js helper, an updated JobDetail.jsx with a confirmation box, an "already applied" state, a log-in message for visitors, no Apply button for recruiters, and 6 frontend tests.

Validation: Ran lint and the frontend tests (all 41 passed), then applied to a job in the browser and confirmed the application was saved in MongoDB Atlas.

Decision: Accepted.

Reflection: AI helped me handle the different user situations and errors, such as applying twice.

Responsible Person: Nazila Hoche Abdi

Task ID: SCRUM-166: Add application status view for users

Tool used: Claude

Purpose of AI Use: To build a page where job seekers can see their applications and status updates over time.

Prompt: Help me implement SCRUM-166, a UI section where users can view the status of their applications.

Response: AI suggested a My Applications page opened from the navbar, showing each application with its current status and history.

What SCRUM-166 needs

The ticket: "Provide a UI section where users can view the status of their submitted job applications and see updates over time." The backend already exists from SCRUM-164 (`GET /api/applications/mine`), and `fetchMyApplications` already exists from SCRUM-165. What's missing is the page itself and a way to get to it.

What it adds

A "My Applications" button in the navbar, shown to job seekers only, next to Profile.

The My Applications page, which shows each application as a card with:

- the job title, company, and location,
- the current status as a coloured badge (Applied blue, Under Review yellow, Interview purple, Offered green, Rejected red),
- "Applied on ..." and, once the status has changed, "Last updated ...",
- a **Status history** section that expands to list every status change with its date. That covers the ticket's "see updates over time".

A Refresh button to load the latest statuses without leaving the page.

Edge cases: an empty state ("You haven't applied to any jobs yet" with a Browse Jobs button), an error message if loading fails, and "Job no longer available" if the recruiter deleted the posting.

The files

All five are full files, so replace or create each one completely:

File	Change
`client\src\components\MyApplications.jsx`	**New:** the page
`client\src\components\MyApplications.test.jsx`	**New:** 6 tests
`client\src\components\Navbar.jsx`	**Replace:** adds the My Applications button
`client\src\components\Navbar.test.jsx`	**Replace:** the existing tests plus 2 new ones at the bottom

AI-Suggested Content: A MyApplications component with colour-coded statuses, an expandable status history, a refresh button, empty and error states, and tests.

Validation: Ran lint and the tests, then changed an application's status as a recruiter and confirmed the job seeker's page showed the update and history.

Decision: Accepted.

Reflection: AI helped me show status changes over time in a clear way and test them from both the recruiter and job seeker side.

Responsible Person: Nazila Hoche Abdi

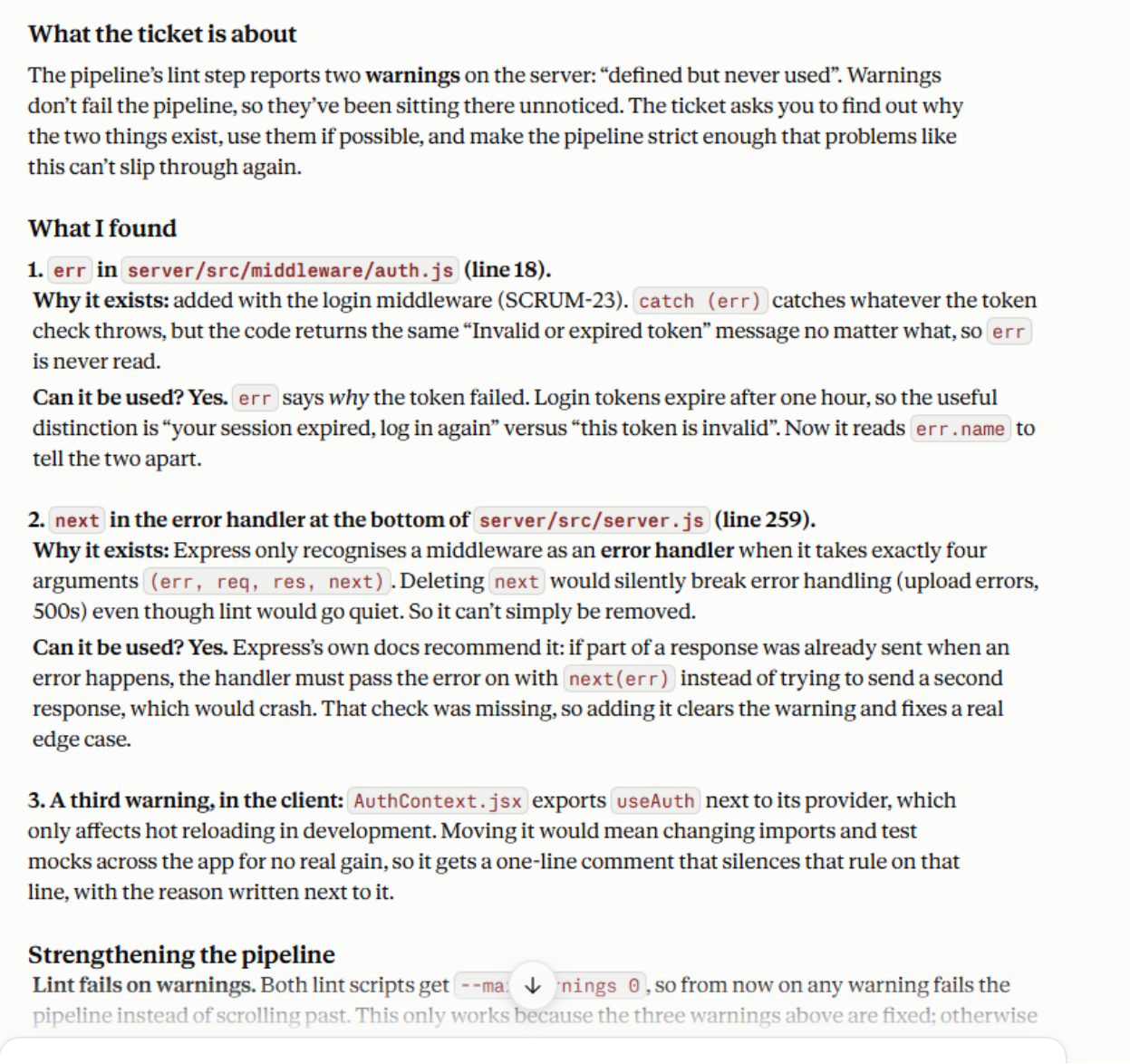
Task ID: SCRUM-211: Cleanup codebase

Tool used: Claude

Purpose of AI Use: To investigate why two unused variables existed and to make the pipeline catch similar issues.

Prompt: Explain this task and guide me through it: two things in the pipeline exist but are never used.

Response: AI found that the unused err in auth.js and the unused next in the error handler could both be used instead of deleted, and that the pipeline was ignoring lint warnings.



What the ticket is about

The pipeline's lint step reports two **warnings** on the server: "defined but never used". Warnings don't fail the pipeline, so they've been sitting there unnoticed. The ticket asks you to find out why the two things exist, use them if possible, and make the pipeline strict enough that problems like this can't slip through again.

What I found

1. `err` in `server/src/middleware/auth.js` (line 18).

Why it exists: added with the login middleware (SCRUM-23). `catch (err)` catches whatever the token check throws, but the code returns the same "Invalid or expired token" message no matter what, so `err` is never read.

Can it be used? Yes. `err` says *why* the token failed. Login tokens expire after one hour, so the useful distinction is "your session expired, log in again" versus "this token is invalid". Now it reads `err.name` to tell the two apart.

2. `next` in the error handler at the bottom of `server/src/server.js` (line 259).

Why it exists: Express only recognises a middleware as an **error handler** when it takes exactly four arguments `(err, req, res, next)`. Deleting `next` would silently break error handling (upload errors, 500s) even though lint would go quiet. So it can't simply be removed.

Can it be used? Yes. Express's own docs recommend it: if part of a response was already sent when an error happens, the handler must pass the error on with `next(err)` instead of trying to send a second response, which would crash. That check was missing, so adding it clears the warning and fixes a real edge case.

3. A third warning, in the client: `AuthContext.jsx` exports `useAuth` next to its provider, which only affects hot reloading in development. Moving it would mean changing imports and test mocks across the app for no real gain, so it gets a one-line comment that silences that rule on that line, with the reason written next to it.

Strengthening the pipeline

Lint fails on warnings. Both lint scripts get `--ma` ↓ `nings 0`, so from now on any warning fails the pipeline instead of scrolling past. This only works because the three warnings above are fixed; otherwise

AI-Suggested Content: Use err to tell expired tokens apart from invalid ones, use next to pass errors on when a response was already sent, make lint fail on warnings (--max-warnings 0), and run the pipeline on all pull requests. Also new tests for the auth middleware.

Validation: Ran lint and all tests locally, then confirmed the pipeline passed on the pull request. When main later failed because of a duplicate lint comment from another merge, I identified the cause and fixed it in a follow-up PR.

Decision: Accepted, with corrections to where the lint comment was placed.

Reflection: AI helped me understand that some unused parameters, like next in Express error handlers, are required, and that a stricter pipeline catches problems earlier.

Responsible Person: Nazila Hoche Abdi

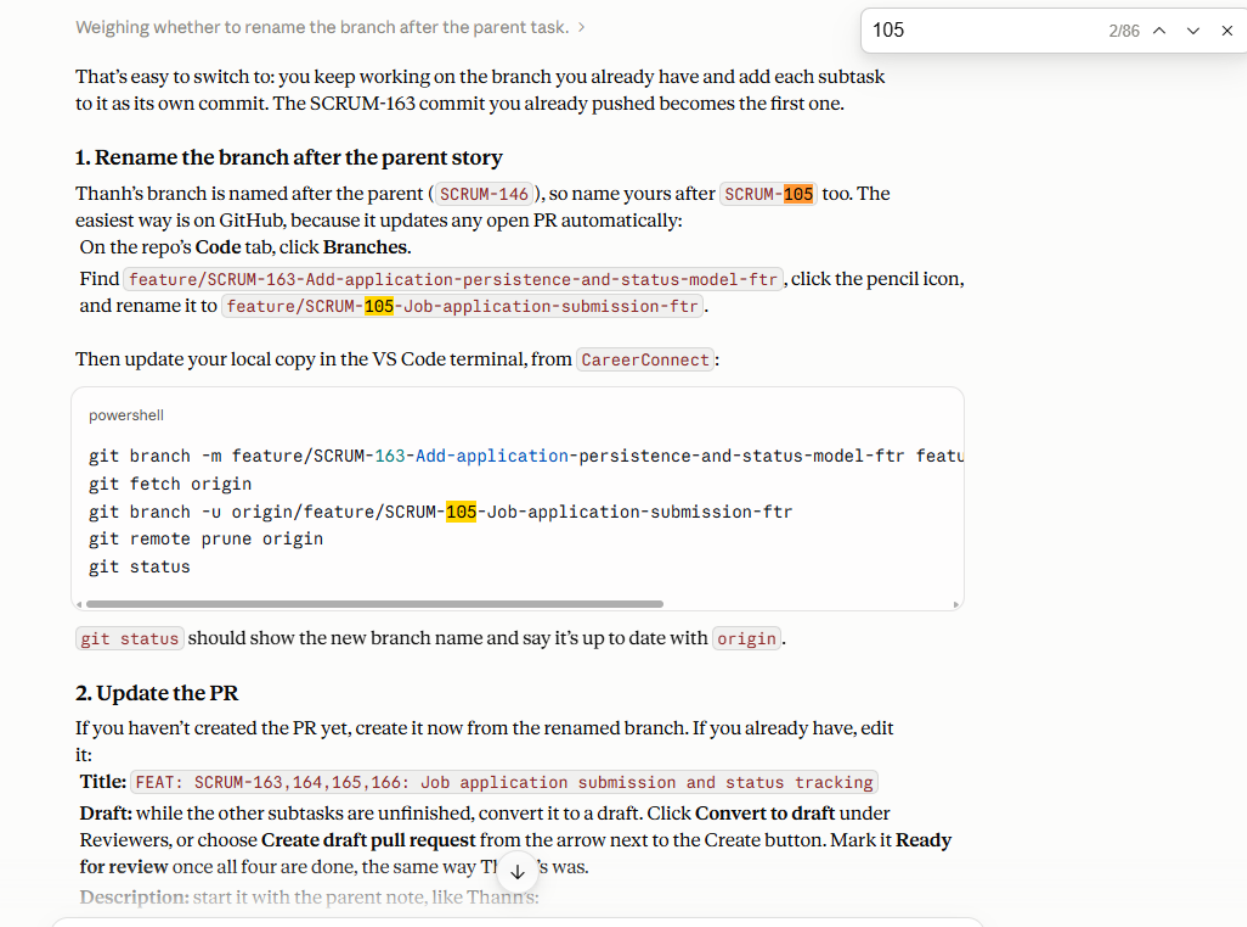
Task ID: SCRUM-105: Git workflow and branch management

Tool used: Claude

Purpose of AI Use: To fix git problems and organize my branches the way the team required.

Prompt: I was asked to work in one branch for all my subtasks that connects to the SCRUM-105 branch. How do I set this up and fix my pull request?

Response: AI explained how to create a subtask branch, reset the parent branch to match main, and open the pull request into the parent branch.



AI-Suggested Content: Git commands for renaming branches, relinking the remote branch, pushing safely, and creating the pull request.

Validation: Checked each step with git status and on GitHub before continuing, and confirmed all commits were on the correct branch.

Decision: Partially accepted and adapted to my team's branch structure.

Reflection: AI helped me understand how branches, remotes, and pull requests relate, and how to recover from mistakes without losing work.

Responsible Person: Nazila Hoche Abdi